Chain of Thought Reinforcement Decoding

Jason Feng
https://x.com/iamjasonfeng
September 21, 2026

Abstract

Chain-of-Thought Reinforcement Decoding (CoTRD) is one of my solutions for the ARC-AGI 3 Kaggle competition. CoTRD is a test-time decoding method designed to increase the influence of an agent's previous reasoning on its next response without weight updates, an external reward model, or a symbolic representation of the task. It measures how the model's output changes when its previous reasoning is removed, then uses that difference during decoding. This paper documents three implementations in the following order: CoTRD (Full CAD), CoTRD Enhanced, and CoTRD Standard. It merely documents CoTRD and does not present or compare scores.

Update notice (November 6, 2026): This revision acknowledges writing assistance from Codex.

Update notice (September 22, 2026): This revision adds citations for Qwen3.8-Flash-Next, the original Tufa Labs Duck harness, and Keith Tyser's Qwen3.8-Flash-Next baseline notebook. It also adds Section 7 to document how the CoTRD notebooks use those works.

1. Introduction

Interactive reasoning agents repeatedly observe an environment, reason about it, and select actions. ARC-AGI 3 evaluates this kind of adaptation in novel interactive environments [4]. An agent's earlier reasoning may remain in context, but its mere presence does not guarantee meaningful influence over subsequent decisions. A model can neglect useful reasoning, weakly attend to it, or replace it with a newly generated but less coherent direction.

CoTRD addresses this problem at decoding time. It treats the previous chain of thought as context whose influence can be measured. Two related contexts are constructed:

- A **conditioned context** containing the previous reasoning.
- A **counterfactual context** from which that reasoning has been removed.

The model's behavior under these contexts reveals how the previous reasoning changes token probabilities. CoTRD then reinforces that measured contribution during the next eligible agent response.

This differs from simply repeating the chain of thought. Repetition modifies the prompt and relies on the model to respond appropriately. CoTRD instead intervenes directly in decoding.

2. Related Work

Context-aware decoding (CAD) contrasts a model's output distributions with and without relevant context, amplifying the contribution of contextual evidence without additional training [1]. CAD reinforces supplied context in general and was introduced to make generation rely more strongly on external evidence. It does **not** specifically reinforce the model's own previous reasoning. CoTRD instead treats an interactive agent's own reasoning from its previous call as the contrasted context and applies the resulting influence to a later observation-action cycle.

Chain-of-Thought Augmentation with Logit Contrast reinforces few-shot chain-of-thought examples supplied by an external source by contrasting chain-of-thought-augmented and unaugmented inputs [2]. CoTRD does not reinforce externally supplied examples or add a newly elicited chain of thought to the same problem instance. It contrasts reasoning generated by the agent during its preceding call, measures that reasoning's effect, and carries the decoding intervention forward to the next environment interaction.

CoTRD is also related to chain-of-thought prompting [3]. Chain-of-thought prompting elicits intermediate reasoning through instructions or demonstrations, whereas CoTRD does not require demonstrations or a prescribed reasoning

format. CoTRD operates after reasoning already exists and modifies decoding to increase that reasoning's influence on subsequent generation.

The distinguishing combination in CoTRD is therefore temporal and intervention-based: the contrasted material is the agent's own reasoning from its previous call, the target is its next interactive response, and the measured difference directly modifies decoding rather than merely being appended to the prompt.

3. CoTRD

Let c denote the shared input for the agent's next model call: the current observation, instructions, retained memory, and any other history preserved by the agent harness, but not the previous call's reasoning being tested. Let r denote that previous reasoning. CoTRD evaluates two otherwise identical model inputs:

$$c^{+} = c \text{ with } r$$

$$c^{-} = c \text{ without } r$$

The difference between their token distributions estimates the contribution of the previous reasoning:

$$\Delta_t = \log P(y_t \mid c^{+}) - \log P(y_t \mid c^{-})$$

A positive value indicates that the previous reasoning increases the token's probability. A negative value indicates that it suppresses the token. CoTRD uses this contrast to modify decoding for the next response.

The method does not assume that the previous reasoning is correct. It reinforces the influence that the reasoning actually exerts, rather than assigning an external semantic reward to it. This makes CoTRD a process-influence mechanism rather than a process-scoring mechanism.

If no previous hidden reasoning is available, the documented implementations use ordinary decoding.

CoTRD uses two implementation families. Full-response decoding computes the conditioned-minus-counterfactual contrast again at every generated token. Probe-based decoding estimates the contrast before the next eligible agent response: it first generates a short continuation with the previous reasoning present, then replays that same continuation with the previous reasoning removed to measure counterfactual token probabilities. Selected differences become a fixed token-bias map for one ordinary tool-enabled request. Replaying known tokens to measure their probabilities is referred to below as teacher forcing.

4. Implementations

4.1 CoTRD (Full CAD)

https://www.kaggle.com/code/iamjasonfeng/cotrd-full-cad

CoTRD (Full CAD) runs two coordinated full-response streams:

- The conditioned stream receives the previous reasoning.
- The counterfactual stream receives the same context without that reasoning.

Before decoding begins, a **synchronization gate** registers both requests and allows both prompts to finish prefill. Neither stream may commit its first generated token until both streams are decode-ready with identical generated-token histories. During generation, if one stream moves ahead, the gate temporarily holds that stream until the other catches up. Genuine history divergence is treated as a synchronization failure rather than silently combining mismatched decoding states. This is an execution-coordination gate, not a confidence or performance gate: the implementation does not predict whether reinforcement will improve the response.

At every decoding step, their logits are combined using a CAD-style rule:

$$z_t = (1 + \alpha)z_t^+ - \alpha z_t^-$$

where zt+ and zt- are the conditioned and counterfactual logits, and α controls reinforcement strength. The documented notebook uses α=0.5.

After the gate admits the pair, both streams use one shared sampled token at every step. Their generated histories therefore remain synchronized while the conditioned-minus-counterfactual contrast is recomputed across the entire response. Coordination continues until completion.

CoTRD (Full CAD) requires approximately twice the model compute of ordinary decoding and places additional demands on request scheduling, KV-cache capacity, and server coordination. Unlike the probe-based implementations, it applies the contrast across the available vocabulary and recalculates reasoning influence after every generated token instead of reducing that influence to a fixed bias map derived from a short probe. The documented notebook disables multi-token prediction, uses a game concurrency of 14, and permits at most four active pairs to accommodate the paired streams. If pairing, synchronization, or verification fails, the request fails open to ordinary decoding when sufficient request time remains.

4.2 CoTRD Enhanced

https://www.kaggle.com/code/iamjasonfeng/cotrd-enhanced

CoTRD Enhanced uses a longer probe and a signed extension of the probe-based approximation.

Its principal changes are:

- Robust contiguous token alignment between generated and teacher-forced probe sequences.
- Signed log-probability differences instead of positive-only measurements.
- Separate selection of positively and negatively affected tokens.
- Bounded reinforcement to prevent extreme decoding distortion.
- Fail-open behavior when reliable alignment or measurement cannot be obtained.

For repeated occurrences of one token ID, CoTRD Enhanced retains the occurrence with the largest absolute measured contribution. It then selects up to 150 positive and 150 negative token IDs and maps their signed contributions to bounded biases. The final agent response is generated through one ordinary request carrying that fixed bias map.

The documented notebook uses a 260-token probe, requires an exact contiguous overlap of at least 16 tokens, discards absolute contributions below 0.08, multiplies retained contributions by 0.5, and clips the resulting biases to the interval [-1.5,1.5]. Measurement or alignment failure produces no bias and leaves the subsequent request on ordinary decoding.

4.3 CoTRD Standard

https://www.kaggle.com/code/iamjasonfeng/cotrd

CoTRD Standard uses two sequential measurement requests:

1. It generates a conditioned probe with the previous reasoning present.
2. It removes the previous reasoning and teacher-forces the conditioned probe to obtain counterfactual log probabilities for the same tokens.

The implementation requires the complete probe token sequence to appear in the teacher-forced prompt. It excludes special tokens, punctuation-only tokens, and a small stopword list; averages the measured contribution across repeated occurrences of each remaining token ID; retains only positive contributions above its threshold; and converts the strongest qualifying token IDs into positive bounded biases. The subsequent agent response remains one ordinary model request using that bias map.

The documented notebook uses a 160-token probe, retains up to 160 token IDs with mean contributions of at least 0.15, and maps them to positive biases between 0.4 and 1.2. Measurement failure produces no bias, while server rejection of the bias triggers an ordinary-decoding retry when time remains. This implementation approximates reasoning-conditioned decoding without requiring two full-response streams.

5. Application to ARC-AGI 3

ARC-AGI 3 presents an interactive setting in which an agent must infer mechanics from visual observations and action consequences. Reasoning developed during one action can remain relevant to later actions, but standard autoregressive inference provides no guarantee that this reasoning will retain sufficient influence.

CoTRD uses the latest previous assistant call's hidden reasoning as the contrastive context. The conditioned and counterfactual paths otherwise receive matching gameplay information. The notebooks permit at most one CoTRD measurement or Full CAD pair per analyzer turn. Because the model request occurs before its tool choice is known, that first request may ultimately produce either an action or an inspection; later inspection continuations in the same analyzer turn use ordinary decoding.

The method operates entirely at test time. It does not fine-tune the model, assign explicit rewards to reasoning steps, or introduce a symbolic world model.

6. Continual Learning and Alignment

The relationship between CoTRD and continual learning should be studied, particularly in settings involving long interaction histories or persistent memory. Research should also evaluate CoTRD's implications for alignment, including whether reinforcing previous reasoning can improve aligned behavior or introduce new risks. These questions require dedicated experiments and are outside the scope of this paper.

7. Implementation Foundations

The documented CoTRD notebooks use Qwen3.8-Flash-Next as their underlying multimodal language model [8]. CoTRD does not modify the model's trained weights. Instead, it changes how the model's own previous hidden reasoning influences decoding during a later interaction.

The notebooks retain the core agent and environment integration provided by the original Tufa Labs Duck harness [7], including the ARC-AGI 3 interaction loop, Python tools, action execution, history handling, and memory infrastructure. CoTRD is implemented as an inference-time extension around that harness rather than as a replacement for it.

Keith Tyser's Duck Qwen3.8-Flash-Next NVFP4 MTP notebook provides the direct Kaggle implementation baseline [6]. It combines the Duck-derived agent with a single-GPU Qwen3.8-Flash-Next serving configuration. The CoTRD notebooks preserve that baseline's model and general gameplay environment while adding their respective conditioned-versus-counterfactual decoding mechanisms and the systems changes required to support them.

8. Limitations

CoTRD cannot determine whether previous reasoning is correct. Stronger influence can preserve useful deductions, but it can also reinforce an incorrect direction.

Probe-based CoTRD only approximates full contrastive decoding. Its effectiveness depends on probe alignment, token selection, reinforcement strength, and whether a short continuation adequately represents the influence of the previous reasoning.

CoTRD (Full CAD) provides a token-synchronous intervention but has substantial systems costs. Paired streams must remain synchronized, and reduced concurrency may be necessary to control memory pressure. These trade-offs can reduce the number of environment interactions completed within a fixed runtime budget.

Finally, ARC-AGI 3 outcomes can vary across runs. Controlled comparisons using matched models, contexts, concurrency, decoding settings, and compute budgets are necessary to isolate the effect of CoTRD.

9. Conclusion

CoTRD is a family of test-time decoding methods intended to make an interactive agent's previous reasoning exert greater influence over its next response. CoTRD (Full CAD) performs token-synchronous contrastive decoding across the complete response. CoTRD Enhanced extends the probe-based measurement procedure while retaining a single gameplay request. CoTRD Standard uses a positive probe-and-bias implementation.

The broader idea is that reasoning history should not merely occupy context. Its contribution can be measured and actively incorporated into decoding.

Acknowledgements

I came up with the CoTRD idea. Codex handled the technical details. Codex also assisted me with writing this paper [5].

References

1. Shi, W., Han, X., Lewis, M., Tsvetkov, Y., Zettlemoyer, L., and Yih, S. W. (2024). *Trusting Your Evidence: Hallucinate Less with Context-aware Decoding*. NAACL 2024. Paper
2. Shim, J., Kruttschnitt, G., Ma, A., Kim, D., Chek, B., Anand, A., Zhu, K., and O'Brien, S. (2024). *Chain-of-Thought Augmentation with Logit Contrast for Enhanced Reasoning in Language Models*. arXiv:2407.03600
3. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., and Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022. Paper
4. ARC Prize Foundation. (2026). *ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence*. Technical report
5. OpenAI. *Codex*. Webpage
6. Tyser, K. (2026). *Duck Qwen3.8 Flash Next NVFP4 MTP*. Kaggle notebook
7. Tufa Labs. (2026). *Tufa Labs Duck Harness: June 30 Milestone Winner*. Kaggle notebook
8. Qwen Team. (2026). *Qwen3.8-Flash-Next*. Model card; Qiu, Z. et al. (2026). *On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability*. arXiv:2608.30320